МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Кемеровский государственный университет» Институт Цифры

ДОМАШНЯЯ РАБОТА №2 ПО ДИСЦИПЛИНЕ

“Параллельные вычисления”

Задача № 6

студентов 3 курса

Сизов Максим Евгеньевич

Грачев Ярослав Евгеньевич

Направление 02.03.02 – Фундаментальная информатика и информационные технологии

Преподаватель:
к-т физ.-мат.наук, доцент
С.В. Стуколов

_____

Работа защищена:
“____”_____2026г.

с оценкой _____

Кемерово 2026

СОДЕРЖАНИЕ

1. Постановка задачи.....3
2. Описание используемых функций.....4
3. Описание программы.....5
4. Реализация.....6
Заключение.....8
Литература.....9

1. Постановка задачи

Задача 6:
Напишите программу, используя блокирующие коммуникационные функции (MPI_Send, MPI_Recv), реализующую алгоритм передачи данных по кольцу. Алгоритм программы можно представить следующим образом: на 0 процессе инициализируется переменная (int a); 0 процесс пересылает переменную a первому процессу; первый, получив значение переменной a, добавляет к нему единицу и отправляет на 2-й процесс. … (size-1) процесс, получив значение переменной a от (size-2) процесса, прибавляет к значению a единицу и отправляет на 0-й процесс.

*Во время выполнения программы соберите трассу параллельного приложения и в отчете приведите скрин визуализации трассы параллельного приложения.

2. Описание используемых функций

2.1 MPI_Send

Синтаксис функции отправки сообщения: int MPI_Send(void* buf, int count, MPI_Datatype datatype, int dest, int tag, MPI_Comm comm)

Входные аргументы: buf - адрес начала расположения пересылаемых данных; count - число пересылаемых элементов; datatype - тип посылаемых элементов; dest - номер процесса-получателя в группе, связанной с коммуникатором comm; tag - идентификатор сообщения; comm - коммуникатор области связи. Функция выполняет посылку count элементов типа datatype сообщения с идентификатором tag процессу с номером dest в области связи коммуникатора comm. Аргумент buf – это то, что требуется отправлять: массив, структура или скалярная переменная. В случае отправки переменной значение count = 1. Следует отметить, что первый аргумент должен быть адресом отправляемых данных. Например, требуется отправить значение переменной a: первым аргументом требуется указать адрес переменной - &a.

2.2 MPI_Recv

Синтаксис функции приема сообщения: int MPI_Recv(void* buf, int count, MPI_Datatype datatype, int source, int tag, MPI_Comm comm, MPI_Status *status)

Входные аргументы: count - максимальное число принимаемых элементов; datatype - тип элементов принимаемого сообщения; source - номер процесса-отправителя; tag - идентификатор сообщения; comm - коммуникатор области связи.

Выходные аргументы: buf - адрес начала расположения принимаемого сообщения; status - атрибуты принятого сообщения. Функция выполняет прием count элементов типа datatype сообщения с идентификатором tag от процесса source в области связи коммуникатора comm.

2.3 Структура MPI_Status

Назначение: структура, в которую функции приёма сообщений (MPI_Recv, MPI_Probe и др.) записывают служебную информацию о принятом сообщении: номер процесса-отправителя, идентификатор (тег) сообщения и код ошибки.
Синтаксис (объявление): MPI_Status stat;

3. Описание программы

Программа реализует алгоритм передачи данных по кольцу с использованием блокирующих коммуникационных функций MPI_Send и MPI_Recv. На нулевом процессе инициализируется целочисленная переменная n, после чего запускается последовательная передача этой переменной по цепочке процессов, образующих замкнутое кольцо. Процесс 0 отправляет начальное значение процессу 1 с тегом 111 и переходит в режим ожидания ответа. Каждый промежуточный процесс, получив значение от предыдущего процесса, увеличивает его на единицу и передаёт следующему — также с тегом 111. Последний процесс в цепочке (с рангом size-1), получив значение от процесса size-2, увеличивает его на единицу и отправляет обратно на нулевой процесс, но уже с другим тегом — 777. Процесс 0 принимает это сообщение и выводит на экран число процессов и итоговое значение переменной. Использование двух различных тегов позволяет логически разделить два потока сообщений: передачу по цепочке «вперёд» и завершающий возврат результата. Разделение логики на ветви if (rank == 0) и else обеспечивает корректную работу всех процессов без взаимных блокировок: нулевой процесс сначала отправляет, а затем принимает, тогда как все остальные процессы, наоборот, сначала принимают, а затем отправляют. Ветвление внутри else по условию rank < size - 1 позволяет последнему процессу отправить сообщение не следующему по цепочке, а обратно нулевому.

4. Реализация

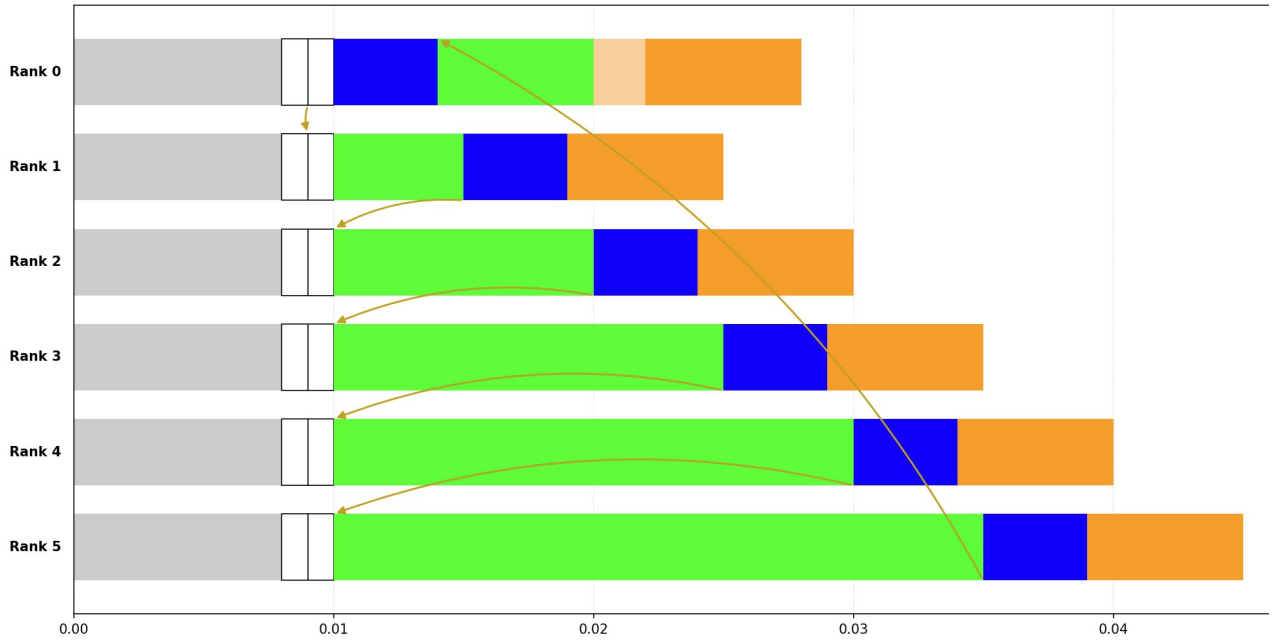
```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
	MPI_Init(&argc, &argv);
	int rank, size, n;
	MPI_Status stat;
	MPI_Comm_rank(MPI_COMM_WORLD, &rank);
	MPI_Comm_size(MPI_COMM_WORLD, &size);

	if (rank == 0) {
		n = 0;
		MPI_Send(&n, 1, MPI_INT, rank + 1, 111, MPI_COMM_WORLD);
		MPI_Recv(&n, 1, MPI_INT, size - 1, 777, MPI_COMM_WORLD,
&stat);
		printf("size: %d | result: %d\n", size, n);
	}
	else {
		MPI_Recv(&n, 1, MPI_INT, rank - 1, 111, MPI_COMM_WORLD,
&stat);
		n += 1;
		if (rank < size - 1) MPI_Send(&n, 1, MPI_INT, rank + 1, 111,
MPI_COMM_WORLD);
		else MPI_Send(&n, 1, MPI_INT, 0, 777, MPI_COMM_WORLD);
	};

	MPI_Finalize();
	return 0;
}
```

```
→  lab2 mpicc 1.c -o first
→  lab2 mpiexec first
size: 6 | result: 5
```


Rank 0
Rank 1
Rank 2
Rank 3
Rank 4
Rank 5
0.00
0.01
0.02
0.03
0.04

Заключение

В ходе выполнения лабораторной работы были изучены и практически применены блокирующие коммуникационные функции MPI типа точка-точка — MPI_Send и MPI_Recv, а также структура MPI_Status, предназначенная для получения служебной информации о принятом сообщении. На примере задачи о передаче данных по кольцу было показано, как организуется последовательный обмен сообщениями между процессами, образующими замкнутую цепочку, и как с помощью тегов можно разделять различные потоки данных внутри одной программы. Были освоены практические приёмы предотвращения дедлоков: поскольку блокирующие функции Send и Recv могут привести к взаимному ожиданию, порядок их вызовов был спроектирован так, чтобы ни один процесс не оказался в состоянии бесконечного ожидания — инициатор сначала отправляет сообщение, а затем принимает ответ, тогда как остальные процессы сначала принимают, а затем отправляют. Дополнительно был освоен приём логирования вызовов MPI с использованием MPI_Wtime: каждый процесс записывал свои события в отдельный лог-файл с метками времени, что позволило впоследствии построить диаграмму Ганта и наглядно визуализировать трассу параллельного приложения. Анализ полученной диаграммы подтвердил, что сообщение действительно последовательно проходит через все процессы по кольцу и возвращается к инициатору, а итоговое значение переменной соответствует ожидаемому. Таким образом, были получены навыки как в области организации обмена сообщениями между процессами, так и в области анализа поведения параллельной программы с помощью трассировки.

Литература

1.Стуколов С.В. Параллельное программирование. Практикум